

Partitioning by Divisor Count

Company: Infosys

Round: Infosys SP / DSE Online Assessment

Year: 2025

Difficulty: Hard

Topic: Dynamic Programming, Two-Pointer / Sliding Window, Prefix Sums, Number Theory

Source: Infosys SP/DSE OA

Problem Description

You are given an array A of N integers. The value of a subsegment is the product of all elements in it. You must count the number of ways to partition the array into consecutive subsegments (covering every element exactly once) such that **every** subsegment's product has **at least X divisors**. Since the count of valid partitions can be very large, return it modulo $10^9 + 7$.

Input Format

- The first line contains an integer N — the number of elements in A .
- The next line contains an integer X — the minimum number of divisors required for each subsegment's product.
- The next N lines each contain an integer $A[i]$.

Output Format

- Return a single integer — the number of valid partitions, modulo $10^9 + 7$.

Constraints

- $1 \leq N \leq 10^5$
- $2 \leq X \leq 10^5$
- $1 \leq A[i] \leq 10^5$

Sample Input / Output

Example 1

Input:

2
2
2
2
2

Output:

2

Example 2

Input:

3
5
2
6
8

Output:

1

Example 3

Input:

5
3
2
2
3
2
2

Output:

3

Explanation

In Example 1, $A = [2, 2]$, $X = 2$. Partition $[2], [2]$: each part's product is 2, which has divisors $\{1, 2\}$ — exactly 2 divisors, valid. Partition $[2, 2]$: product is 4, which has more than 2 divisors, also valid. Total: 2 ways.

In Example 2, $A = [2, 6, 8]$, $X = 5$. Only taking the whole array as one segment works: $2 \times 6 \times 8 = 96$, whose divisor count exceeds 5. No smaller partition works.

In Example 3, $A = [2, 2, 3, 2, 2]$, $X = 3$. Three valid partitions: $[2, 2, 3][2, 2]$, $[2, 2][3, 2, 2]$, and $[2, 2, 3, 2, 2]$ as a single segment.

Approach to Solve This Problem:

The central difficulty in this problem is verifying, for any subsegment, whether its product has at least X divisors — without ever computing the product itself, since it would be far too large to store or work with directly. The solution instead relies on tracking prime factorization data incrementally as a sliding window expands and contracts.

Observation 1 — Divisor count only increases as a segment grows.

The number of divisors of a number is determined by its prime factorization: if a number equals $p_1^{e_1} \times p_2^{e_2} \times \dots \times p_k^{e_k}$, its divisor count is $(e_1 + 1)(e_2 + 1)\dots(e_k + 1)$. Extending a subsegment by including one more element can only add to these exponents, never remove from them. As a result, the divisor count of a subsegment's product can only stay the same or increase as the subsegment grows — it never decreases.

Observation 2 — This monotonic behavior allows a two-pointer (sliding window) technique.

Because divisor count grows monotonically with segment length, for each right endpoint r , there is a well-defined minimum starting position from which the segment first becomes valid (satisfies divisor count $\geq X$). As r increases, this minimum starting position also moves forward, never backward. This is the same structural property used in the classic "smallest subarray with sum at least K " problem, and it allows the left boundary of the window to advance monotonically across the entire array, giving an efficient linear-time scan overall.

Observation 3 — The counting itself is a straightforward prefix-sum DP.

Let $dp[i]$ represent the number of valid ways to partition the first i elements of the array. For a given right endpoint r , once the earliest valid starting position (call it $bestL$) is identified, every starting position from 1 up to $bestL$ is also guaranteed valid, due to the monotonic property above. This means $dp[r]$ can be computed directly as the sum of $dp[j]$ for all j from 0 to $bestL - 1$, which is simply a lookup into a running prefix-sum array — no repeated summation is needed.

Putting these observations together, the solution proceeds as follows:

1. **Precompute a smallest-prime-factor (SPF) sieve** for all values up to 10^5 , so that any element of A can be factorized quickly, in time proportional to its number of prime factors.
2. **Maintain a running record of prime exponents** for the elements currently inside the sliding window, along with the window's current divisor count, updating both incrementally as elements enter or leave the window.
3. **For each right endpoint r, expand the window** by factoring in $A[r]$, then **shrink the window from the left** for as long as the divisor count requirement continues to hold, recording the furthest left position reached as bestL.
4. **Compute $dp[r]$ using the prefix-sum array**: if a valid window was found, $dp[r]$ equals the prefix sum up to $bestL - 1$; otherwise $dp[r]$ is 0.
5. **Update the prefix-sum array** to include $dp[r]$, and proceed to the next r.
6. **Return $dp[N]$ modulo $10^9 + 7$** as the final answer.

Java Solution:

```
import java.util.*;

public class Solution {
    static final int MOD = 1_000_000_007;
    static final int MAX_A = 100_000;
    static int[] spf = new int[MAX_A + 1];
    static boolean sieveBuilt = false;

    private static void buildSieve() {
        if (sieveBuilt) return;
        for (int i = 2; i <= MAX_A; i++) {
            if (spf[i] == 0) {
                for (int j = i; j <= MAX_A; j += i) {
                    if (spf[j] == 0) {
                        spf[j] = i;
                    }
                }
            }
        }
        sieveBuilt = true;
    }

    public static int getCount(int N, int X, List<Integer> A) {
        buildSieve();

        Map<Integer, Integer> exponent = new HashMap<>();
        long divisorCount = 1;

        long[] dp = new long[N + 1];
        long[] prefixSum = new long[N + 1];
        dp[0] = 1;
        prefixSum[0] = 1;

        int l = 1;

        for (int r = 1; r <= N; r++) {
            int val = A.get(r - 1);
            while (val > 1) {
                int p = spf[val];
                int cnt = 0;
                while (val % p == 0) {
                    val /= p;
                    cnt++;
                }
                int oldExp = exponent.getOrDefault(p, 0);
                divisorCount = divisorCount / (oldExp + 1) * (oldExp + cnt + 1);
                exponent.put(p, oldExp + cnt);
            }
            dp[r] = divisorCount >= X ? prefixSum[r - 1] : 0;
            prefixSum[r] = (prefixSum[r - 1] + dp[r]) % MOD;
        }

        return dp[N] % MOD;
    }
}
```

```

int bestL = -1;

while (l <= r && divisorCount >= X) {
    bestL = l;

    int leftVal = A.get(l - 1);
    while (leftVal > 1) {
        int p = spf[leftVal];
        int cnt = 0;
        while (leftVal % p == 0) {
            leftVal /= p;
            cnt++;
        }
        int oldExp = exponent.get(p);
        int newExp = oldExp - cnt;
        divisorCount = divisorCount / (oldExp + 1) * (newExp + 1);
        if (newExp == 0) {
            exponent.remove(p);
        } else {
            exponent.put(p, newExp);
        }
    }
    l++;
}

if (bestL != -1) {
    dp[r] = prefixSum[bestL - 1];
} else {
    dp[r] = 0;
}

prefixSum[r] = (prefixSum[r - 1] + dp[r]) % MOD;
}

return (int) (dp[N] % MOD);
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int N = sc.nextInt();
    int X = sc.nextInt();

    List<Integer> A = new ArrayList<>();
    for (int i = 0; i < N; i++) {
        A.add(sc.nextInt());
    }

    int result = getCount(N, X, A);
    System.out.println(result);
}

```

Solution:

- 1. Build the sieve once:** buildSieve() precomputes the smallest prime factor of every integer up to 100,000, allowing any element of A to be factorized efficiently later. This is done only once, regardless of the size of the input.
- 2. Initialize the base case:** dp[0] is set to 1, representing the single trivial way to partition an empty prefix, and prefixSum[0] is initialized to match.
- 3. Expand the window to include A[r]:** For each right endpoint r, the current element is factorized using the sieve. For every prime found, the window's divisor count is updated by removing the contribution of its previous exponent and inserting the contribution of its new, larger exponent.
- 4. Shrink the window from the left where possible:** While the current window still satisfies the divisor count requirement, the left boundary bestL is updated to the current position, and the element at the left edge is removed from the window's factorization (its exponents subtracted and the divisor count adjusted accordingly) before advancing the left pointer. This continues until the window can no longer be shrunk without falling below the required divisor count.

5. **Determine $dp[r]$:** If a valid left boundary $bestL$ was found during this process, $dp[r]$ is set to $prefixSum[bestL - 1]$, representing the total number of valid partitions of everything before the earliest valid segment. If no valid boundary exists at all, $dp[r]$ remains 0.
6. **Update the prefix sum:** $prefixSum[r]$ is updated to include $dp[r]$, modulo $10^9 + 7$, so that it is ready for use in later iterations.
7. **Return the final result:** After processing all elements, $dp[N]$ holds the total number of valid ways to partition the entire array, and this value is returned modulo $10^9 + 7$.

Complexity Analysis:

Time Complexity: $O(MAX_A \log \log MAX_A + N \log(MAX_A))$

The sieve construction takes $O(MAX_A \log \log MAX_A)$ time (built once regardless of N). During the main scan, each element is factored into primes at most twice — once when added to the window and once when removed — each taking $O(\log A[i])$ time. Since the left pointer l only ever moves forward across the entire scan (never resets backward), the total work across all N iterations is $O(N \log(MAX_A))$.

Space Complexity: $O(MAX_A + N)$

The SPF sieve requires $O(MAX_A)$ space, and the dp and $prefixSum$ arrays each require $O(N)$ space. The exponent map holds only the primes currently present in the sliding window, which is bounded by a small constant (at most about 6 distinct primes, since the product of the first 7 primes already exceeds 10^5).

Combined Space Complexity: $O(MAX_A + N)$